



北京航空航天大学
BEIHANG UNIVERSITY

机器人导航python实践(入门)

Lecture6-路径规划 part1

北航 国新院 实验实践课
智能系统与人形机器人国际研究中心



教师： 欧 阳 老 师



邮 箱： ouyangkid@buaa.edu.cn



学 期： 2025年秋季

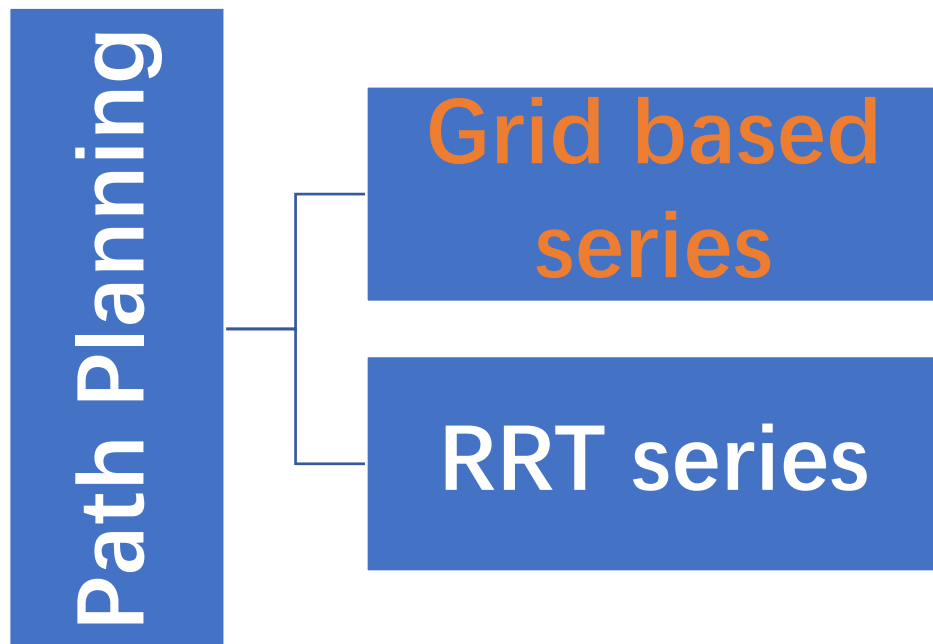
目录

Contents

01 课程内容安排

02 Grid based Search

03 RRT series



路径规划（Path Planning）指机器人搜索前往目标点的可行且高效路径的能力。该路径需依据机器人的运动模型和障碍物位置满足特定约束条件，并对部分目标函数进行优化，例如到达目标点的时间、与障碍物的距离等。

基于网格地图的算法，路径搜索和规划的空间通常被定义为2D数据结构；

而搜索树方法则利用树结构提升表达和计算的效率。

目录

Contents

01 课程内容安排

02 Grid based Search

03 RRT series

目录

Contents

01 Breadth/ Depth First Search

02 Dijkstra algorithm

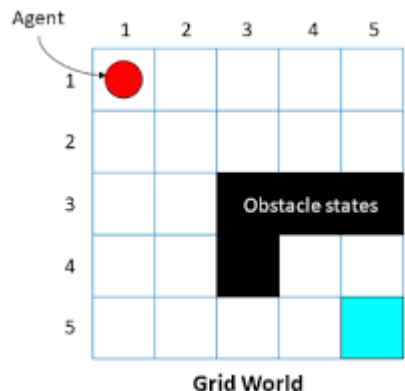
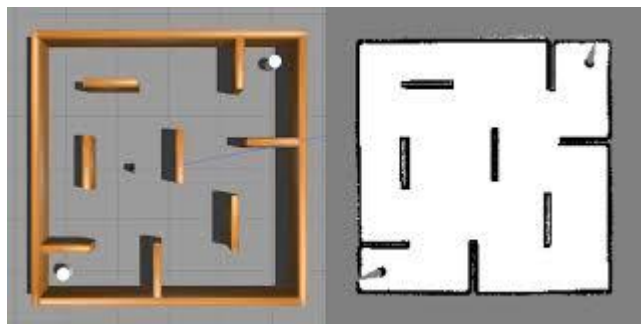
03 A* algorithm

04 D* algorithm

05 Potential Field algorithm

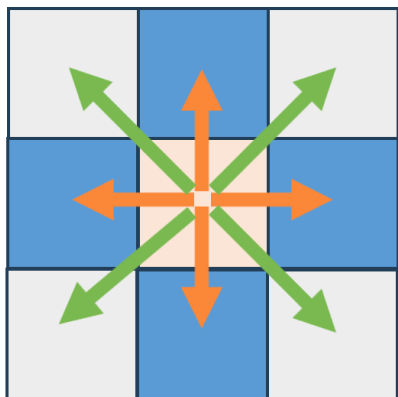
Part 1 | 建图—知识回顾

栅格地图



实际环境→感知结果→坐标系映射

栅格图下的机器人简化运动模型



```
@staticmethod
def get_motion_model():
    return [
        [1, 0, 1],
        [0, 1, 1],
        [-1, 0, 1],
        [0, -1, 1],
        [-1, -1, math.sqrt(2)],
        [-1, 1, math.sqrt(2)],
        [1, -1, math.sqrt(2)],
        [1, 1, math.sqrt(2)]
    ]
```

- 以栅格地图模拟对环境的感知，通过坐标系映射、分辨率配置，简化整体计算的复杂度。
- 基于网格到障碍物的距离，与机器人尺寸，判断通过性。
- 仅未被障碍物占据的栅格参与路径遍历计算。

- 机器人单次能选择8个不同的方向，每种情形下行进的距离有两种，分别为1倍和 $\sqrt{2}$ 倍的栅格边长（用于评估移动代价）。
- 机器人尺寸 $\leq$ 单位栅格：考虑机器人的移动特性
- 实际上机器人的**运动模型顺序**，将影响搜索顺序（如左侧代码：总是先探索右上左下…）。

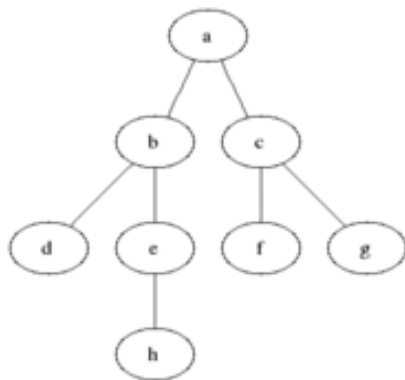
Part 1 | Breadth First Search & Depth First Search

候选优先搜索+路径规划：

- 1.搜索负责度量路径的距离（代价）、障碍物规避
- 2.路径规划负责选择**总距离**最小的路径

» 广度优先搜索 (BFS) 路径规划

BFS是从**根节点**开始，沿着树的**宽度**遍历树的节点（**层优先**）。如果所有节点均被访问，则算法中止。

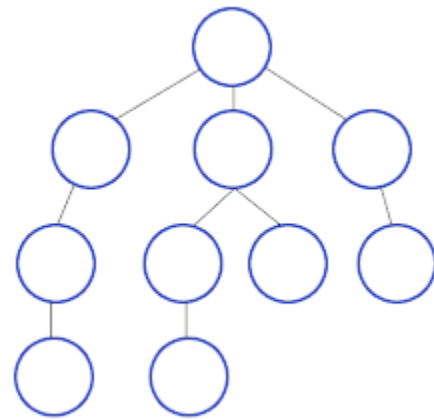


Open-closed集：所有因为展开节点而得到的子节点都会被加进一个先进先出的队列中。一般的实现里，其邻居节点尚未被检验过的节点会被放置在一个被称为 **open** 的容器中（例如队列或是链表），而被检验过的节点则被放置在被称为 **closed** 的容器中。

» 深度优先搜索 (DFS) 路径规划

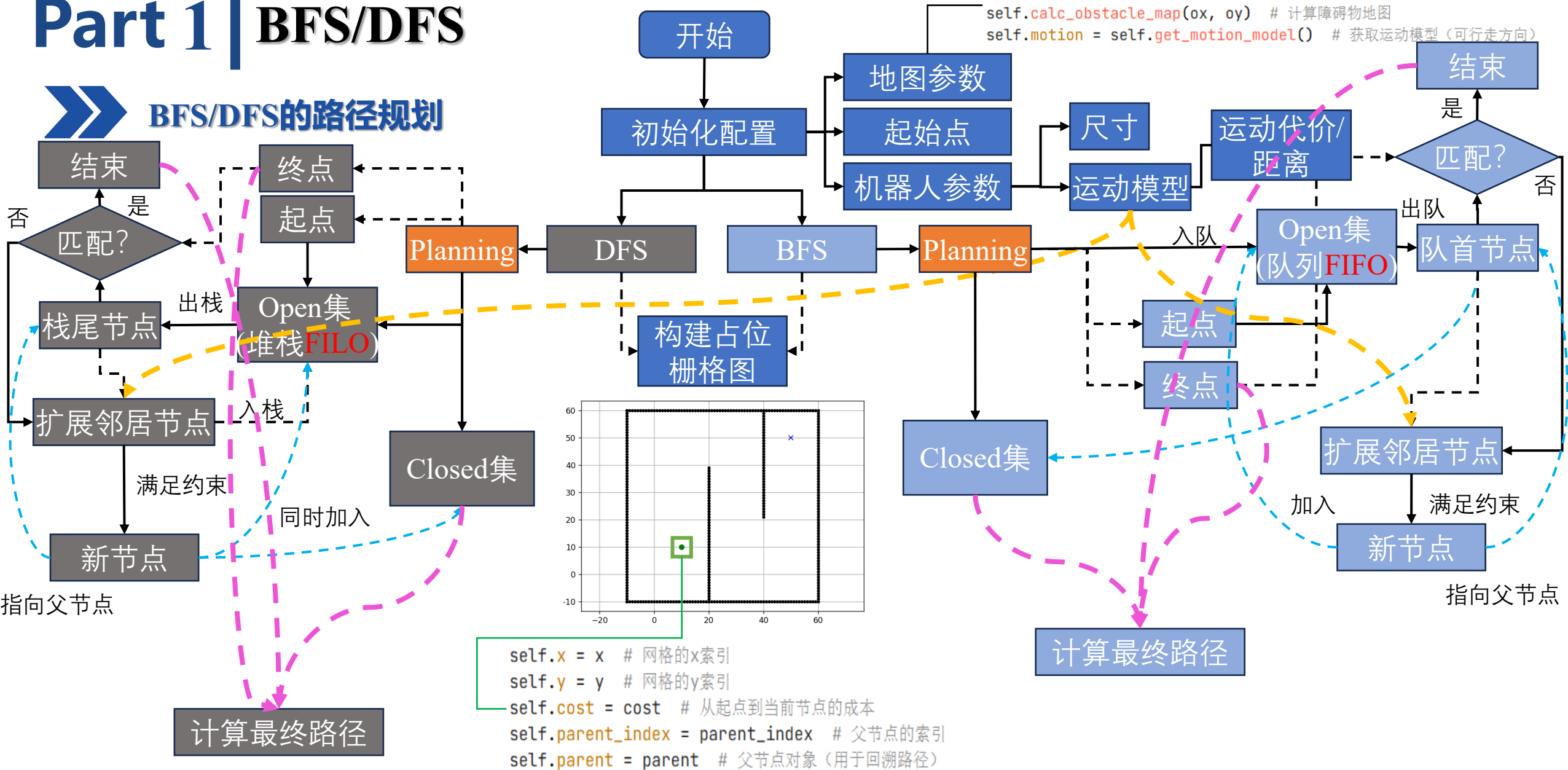
DFS会在回溯并扩展其他节点之前尽可能深入地探索节点分支。这一过程一直进行到已发现从源节点可达的所有节点为止。

在前面的迷宫生成算法中，使用过**DFS**。



Part 1 | BFS/DFS

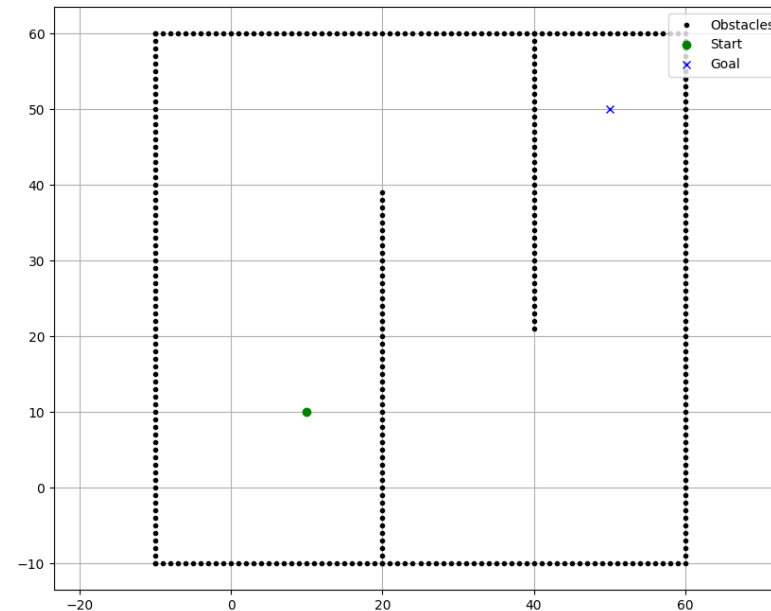
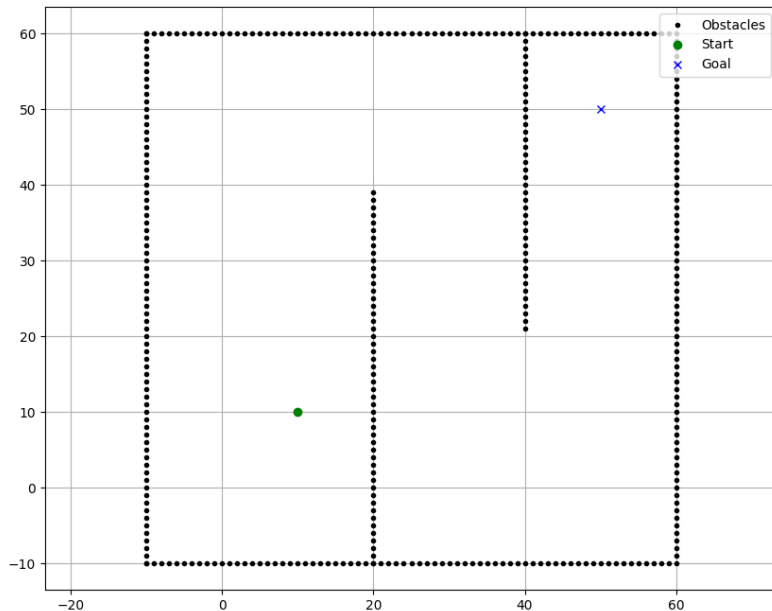
BFS/DFS的路径规划



Part 1 | BFS/DFS



BFS/DFS的路径规划



实验要求:

1. 分析起始点位置;
 2. 运动模型;
 3. 地图场景;
- 对算法的影响。

差别	深度优先搜索 (DFS)	广度优先搜索 (BFS)
节点选取逻辑	从open_set的末尾取节点 (模拟栈的 LIFO 特性)	从open_set的头部取节点 (模拟队列的 FIFO 特性)
closed_set更新时机	扩展节点时, 将节点同时加入open_set和closed_set	处理当前节点后, 将其加入closed_set, 扩展节点时仅检查是否在closed_set和open_set中

目录

Contents

01 Breadth/ Depth First Search

02 Dijkstra algorithm

03 A* algorithm

04 D* algorithm

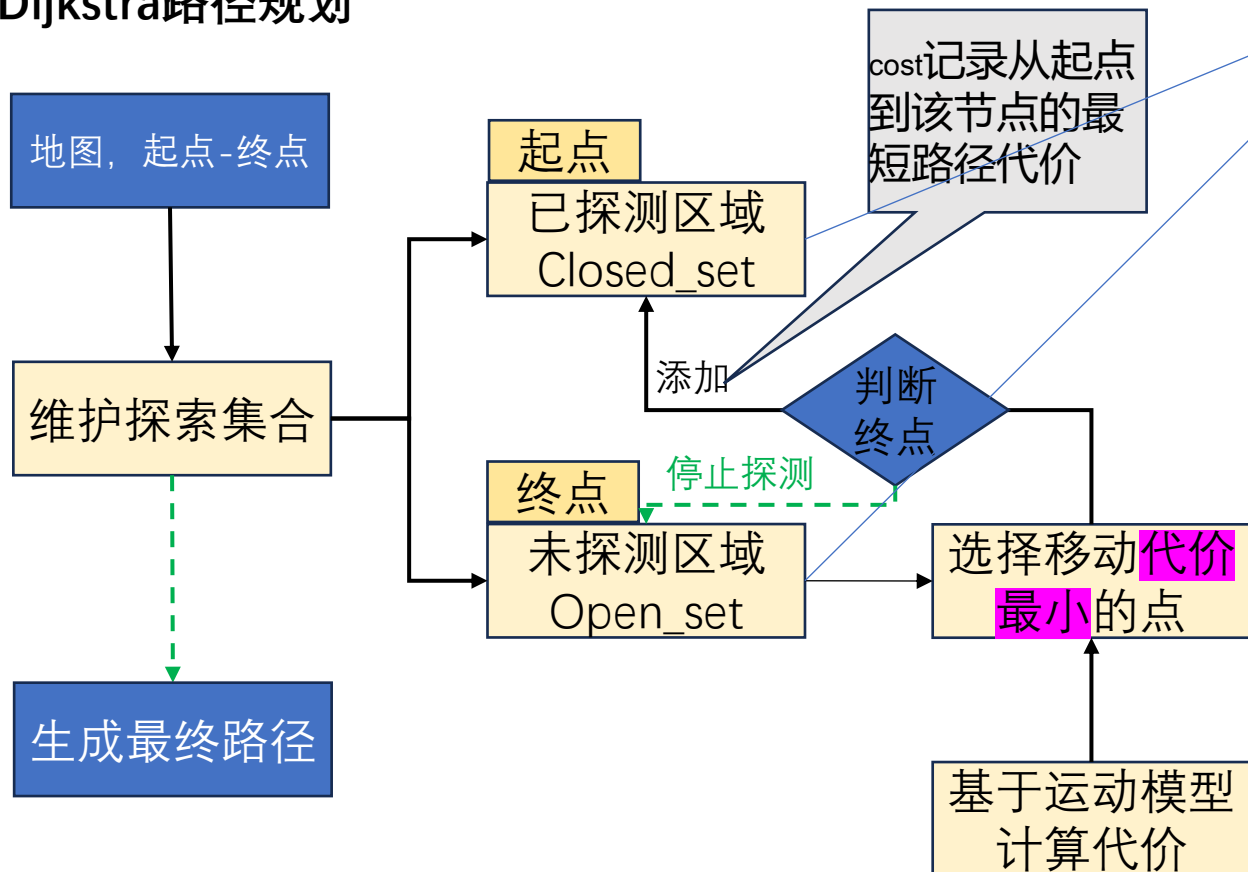
05 Potential Field algorithm

Part 2 | Dijkstra



基于网格地图的Dijkstra

Dijkstra路径规划



```
class Node:
    解释 | 添加注释 | X
    def __init__(self, x, y, cost, parent_index):
        self.x = x
        self.y = y
        self.cost = cost
        self.parent_index = parent_index
```

两个集合均采用Node数据结构来保存栅格地图但是起到了类似链表的功能

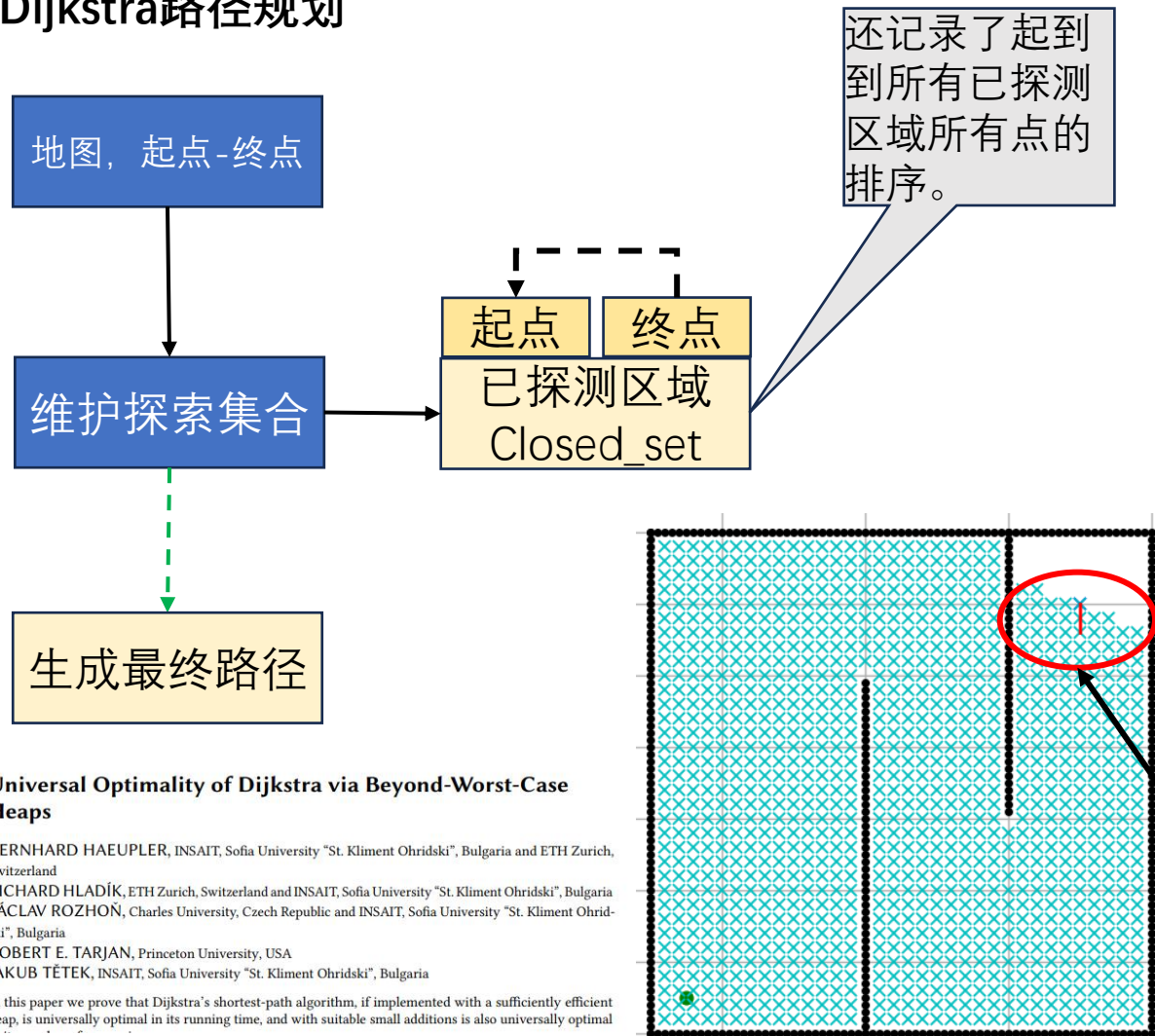
```
node = self.Node(current.x + move_x,
                  current.y + move_y,
                  current.cost + move_cost, c_id)
```

Part 2 | Dijkstra



基于网格地图的Dijkstra

Dijkstra路径规划



Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps

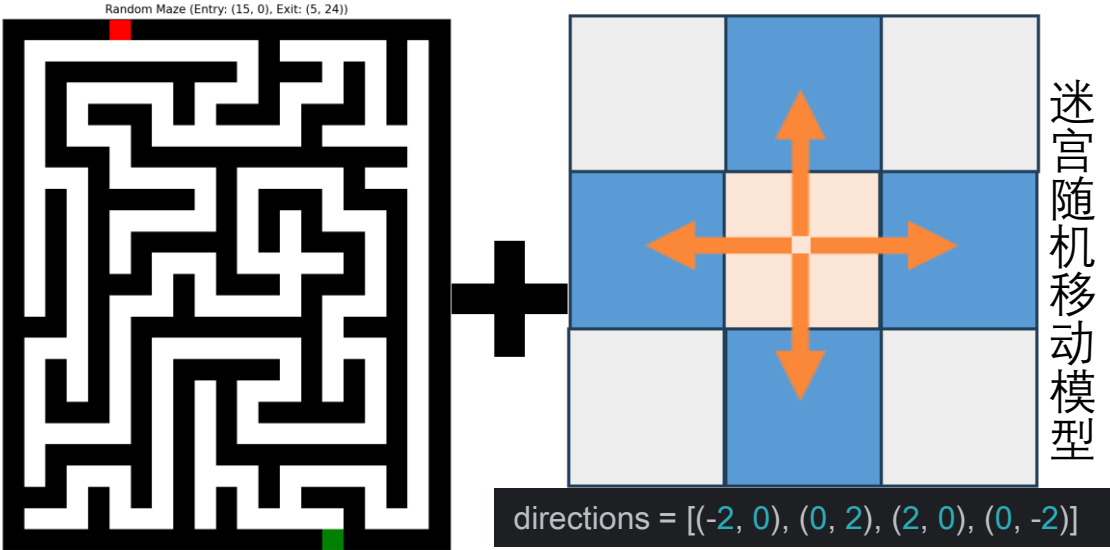
BERNHARD HAEUPLER, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria and ETH Zurich, Switzerland
RICHARD HLADÍK, ETH Zurich, Switzerland and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria
VÁCLAV ROZHON, Charles University, Czech Republic and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria
ROBERT E. TARJAN, Princeton University, USA
JAKUB TĚTEK, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria

In this paper we prove that Dijkstra's shortest-path algorithm, if implemented with a sufficiently efficient heap, is universally optimal in its running time, and with suitable small additions is also universally optimal

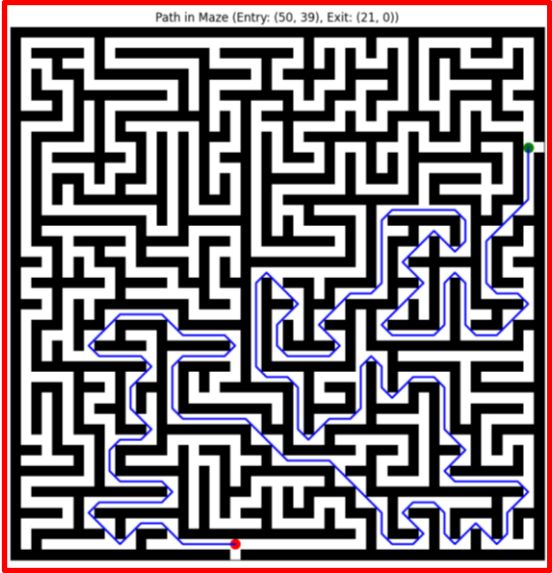
```
def calc_final_path(self, goal_node, closed_set):  
    # 初始化路径列表, 首先加入目标节点的坐标  
    rx, ry = [self.calc_position(goal_node.x, self.min_x)], [  
        self.calc_position(goal_node.y, self.min_y)]  
  
    # 获取目标节点的父节点索引 (指向路径中的前一个节点)  
    parent_index = goal_node.parent_index  
  
    # 循环回溯父节点链, 直到找到起点 (parent_index == -1)  
    while parent_index != -1:  
        # 从closed_set中获取父节点对象  
        n = closed_set[parent_index]  
  
        # 将父节点的坐标添加到路径列表中  
        rx.append(self.calc_position(n.x, self.min_x))  
        ry.append(self.calc_position(n.y, self.min_y))  
  
        # 更新父节点索引, 继续向上回溯  
        parent_index = n.parent_index  
  
    # 返回路径列表 (注意: 此时路径是从终点到起点的顺序, 需要在调用处反转)  
    return rx, ry
```

Part 2 | Dijkstra

基于网格地图的Dijkstra



+ Dijkstra =



差异	DFS	BFS	Dijkstra
节点选取策略	Open_set末尾节点	Open_set队首节点	Open_set中cost最小节点
Node类	父节点索引+对象	父节点索引+对象	父节点索引(回溯closed_set)
Open-closed	不在closed_set, 两个集合都加入	不在open和closed时, 仅加入open	从open移动至closed, 再拓展节点, 基于cost更新open

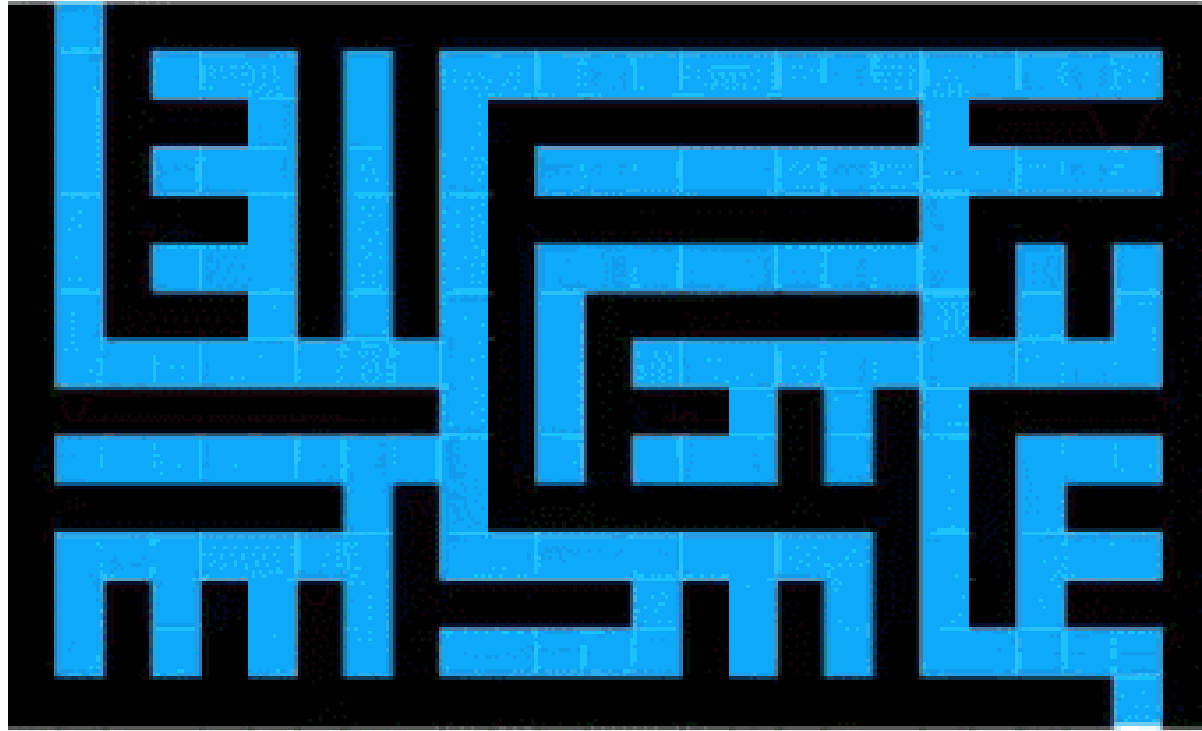
实验要求: 1. 在随机迷宫中, 测试和对比DFS和BFS, 分析它们Dijkstra的差异。

Part 2 | Dijkstra



迷宫的拓扑求解

START



GOOL

目录

Contents

01 Breadth/ Depth First Search

02 Dijkstra algorithm

03 A* algorithm

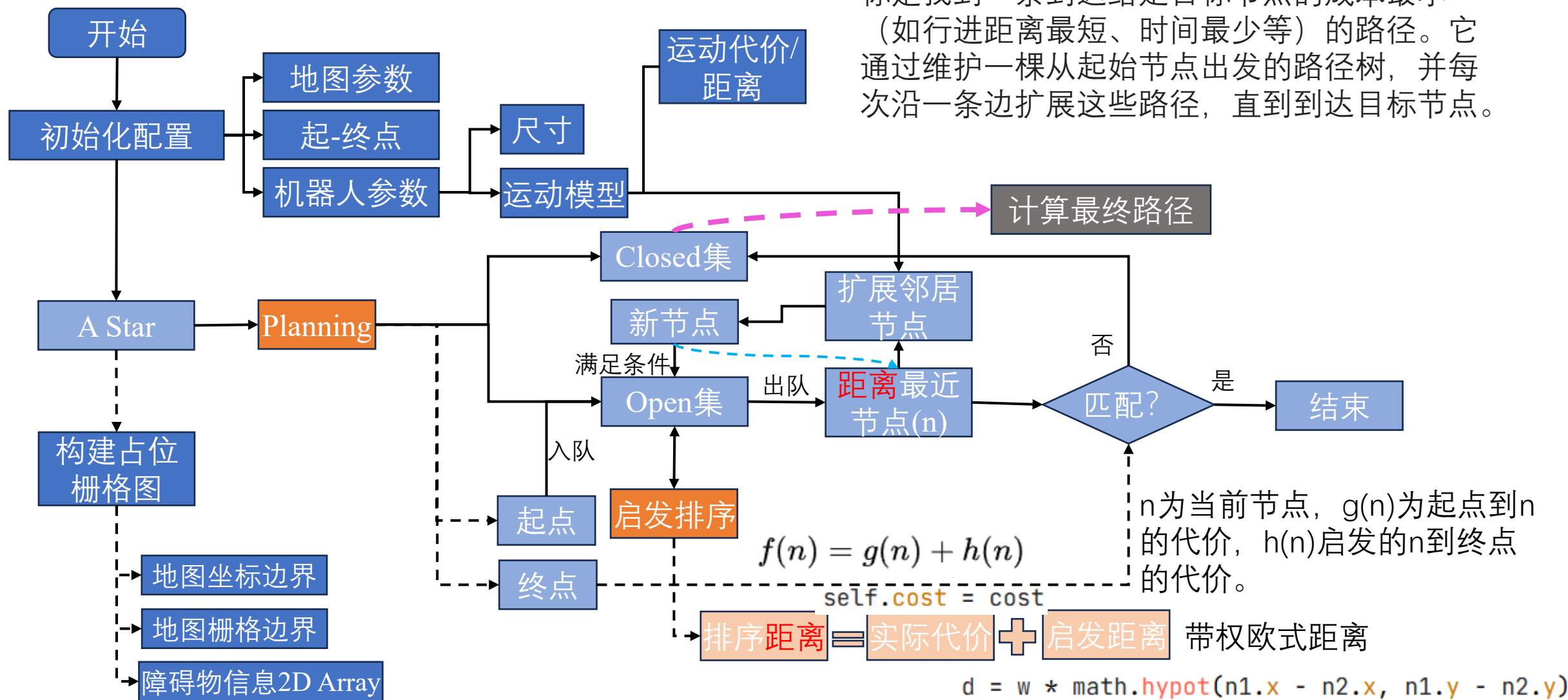
04 D* algorithm

05 Potential Field algorithm

Part 3 | A*



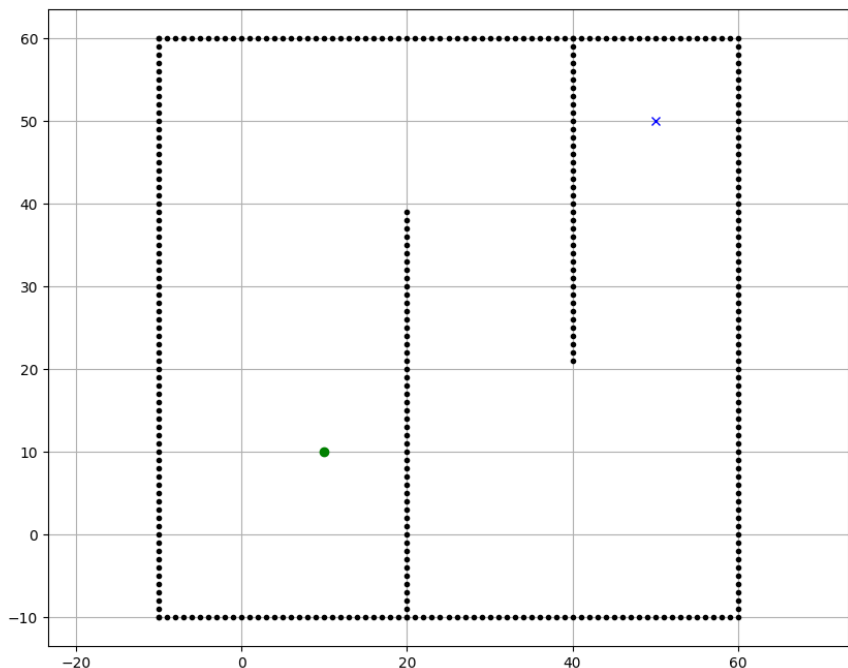
A*是基于信息的搜索算法，从起点出发，其目标是找到一条到达给定目标节点的成本最小（如行进距离最短、时间最少等）的路径。它通过维护一棵从起始节点出发的路径树，并每次沿一条边扩展这些路径，直到到达目标节点。



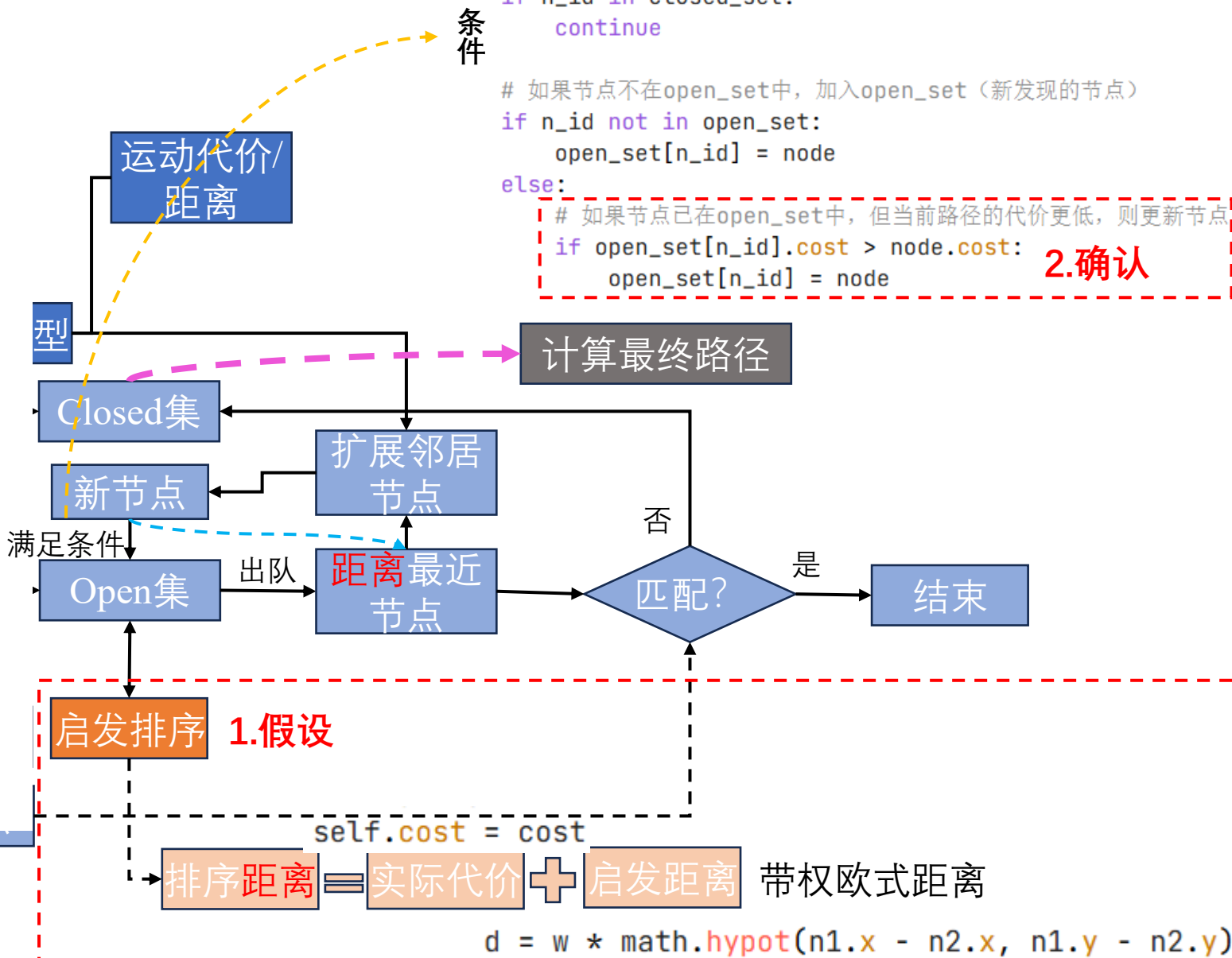
Part 3 | A*



开始



- 地图栅格边界
- 障碍物信息2D Array



目录

Contents

01 Breadth/ Depth First Search

02 Dijkstra algorithm

03 A* algorithm

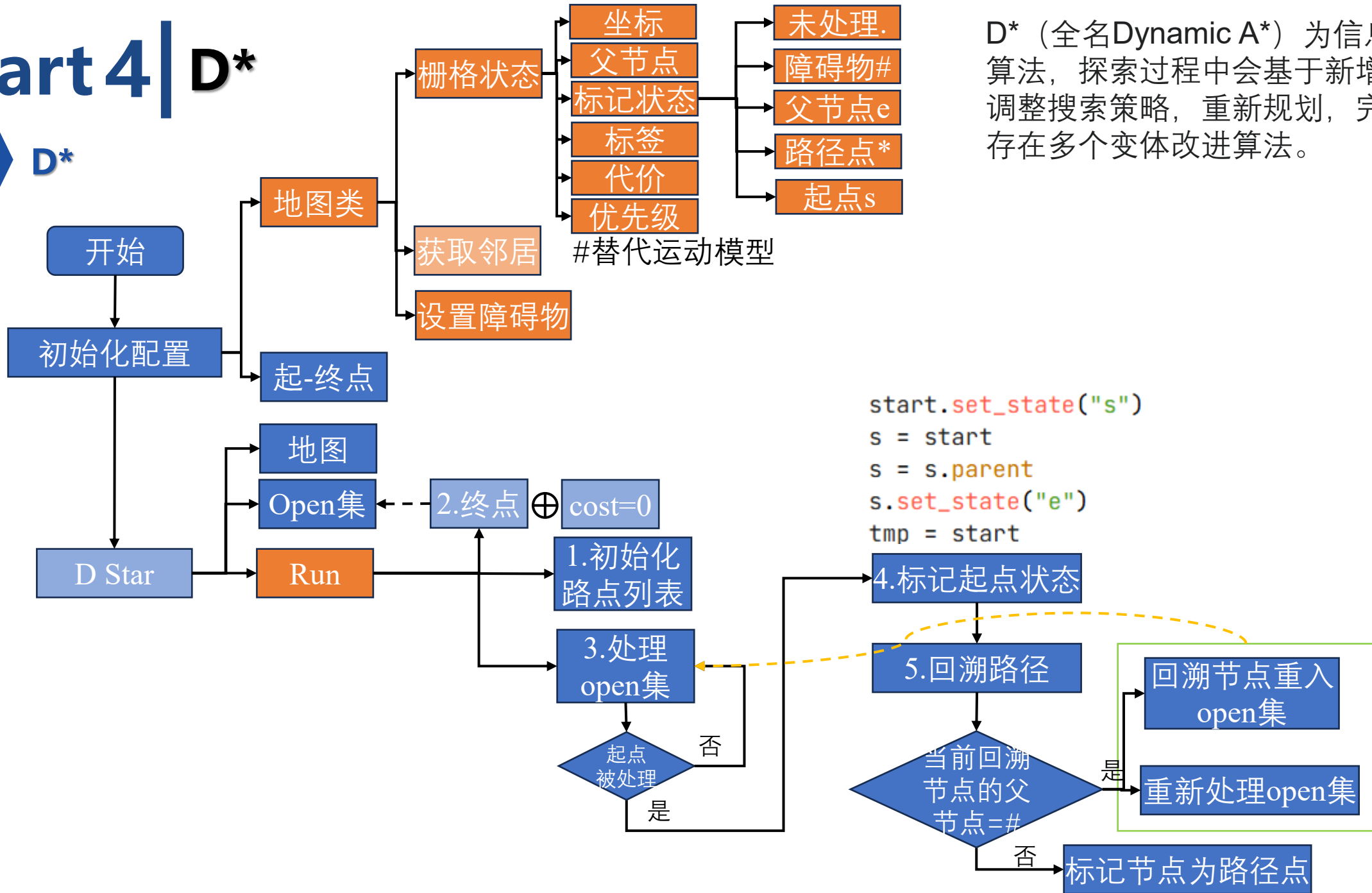
04 **D* algorithm**

05 Potential Field algorithm

Part 4 | D*

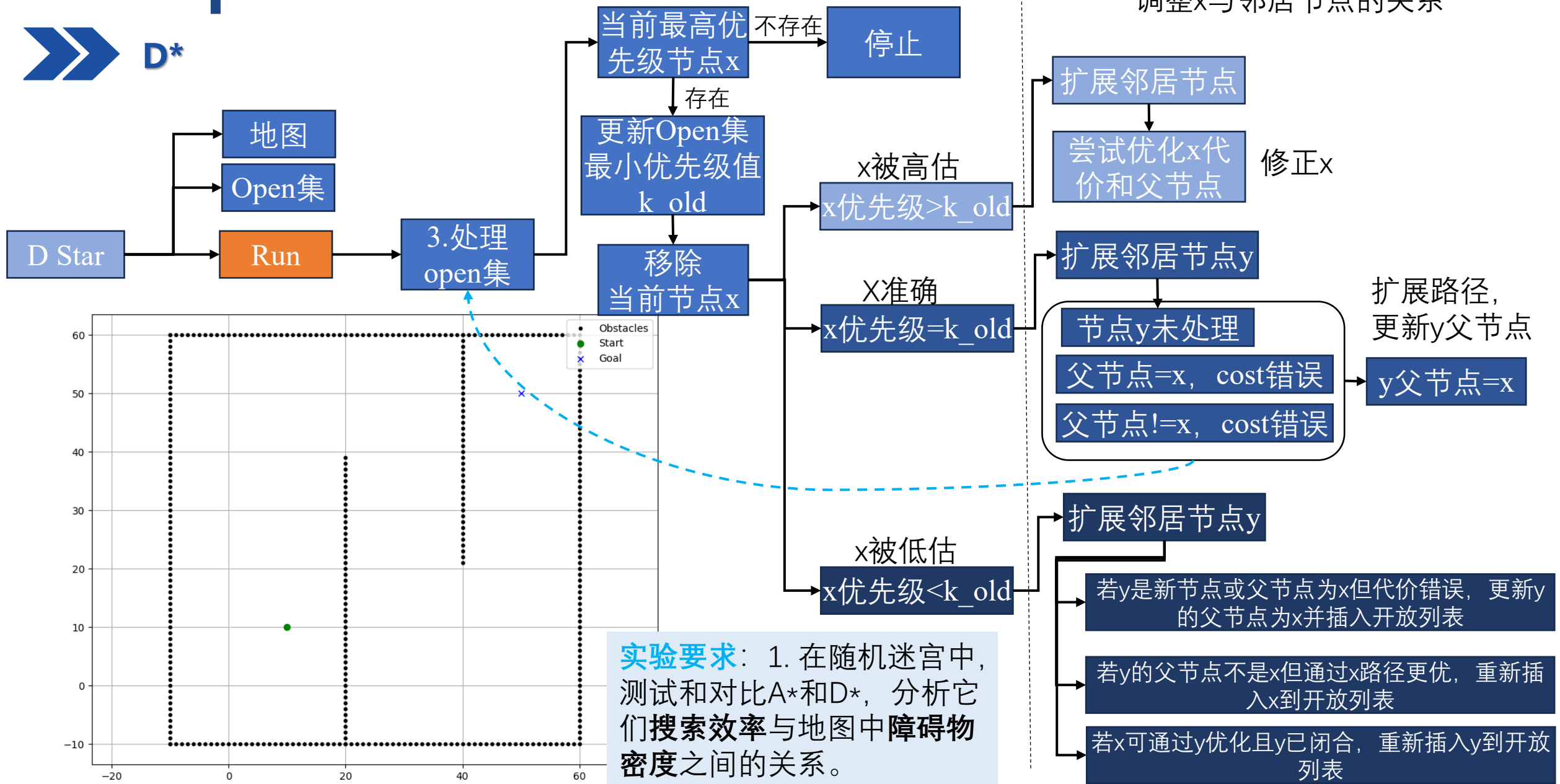
» D*

D* (全名Dynamic A*) 为信息增量搜索算法，探索过程中会基于新增加的信息调整搜索策略，重新规划，完成搜索。存在多个变体改进算法。



```
start.set_state("s")
s = start
s = s.parent
s.set_state("e")
tmp = start
```

Part 4 | D*



目录

Contents

01 Breadth/ Depth First Search

02 Dijkstra algorithm

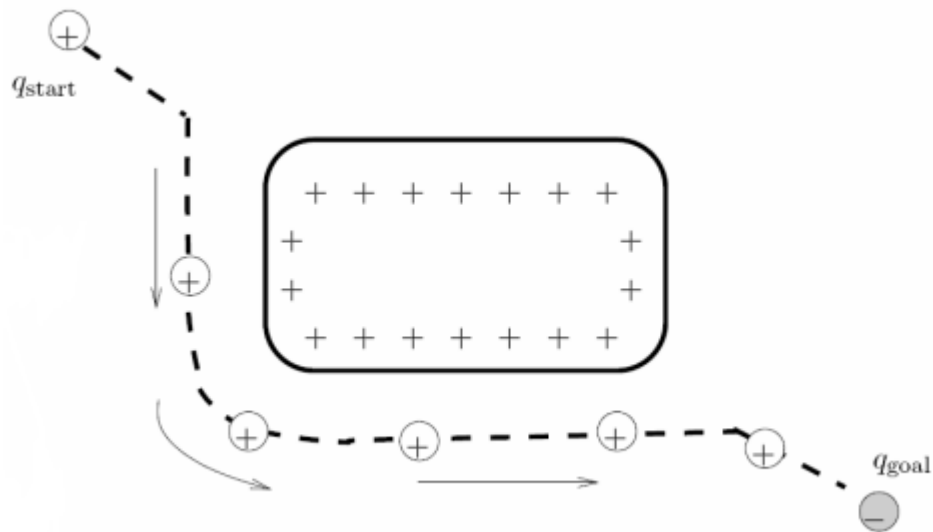
03 A* algorithm

04 D* algorithm

05 Potential Field algorithm

Part 5 | Potential Field algorithm

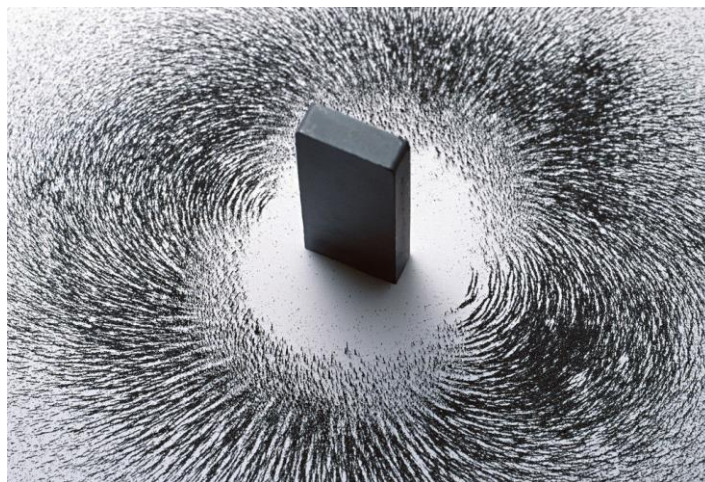
势场导航



势场法路径规划的基本原理：

机器人是正极（任意起点），终点是负极。
障碍物和机器人同为正极，互斥。

- 目标点对机器人产生"引力"，吸引机器人向其移动
- 障碍物对机器人产生"斥力"，排斥机器人远离它们
- 机器人的运动方向由引力和斥力的合力决定



描述势场：

1. 场有个大小，需要描述计算影响的范围；
2. 通常情况下引力场中的引力的作用是相互、对等的。但是在程序算法中，可以设置不同，如希望机器人碰撞避免，可以将斥场增益系数加大。

```
KP = 5.0 # 引力势场增益，用于控制引力的强度
ETA = 100.0 # 斥力势场增益，用于控制斥力的强度
AREA_WIDTH = 30.0 # 势场计算区域宽度 [单位：米]，超出该范围的区域不计算势场
# 用于检测路径振荡的历史位置数量（记录最近多少个位置来判断是否往复运动）
OSCILLATIONS_DETECTION_LENGTH = 3
```

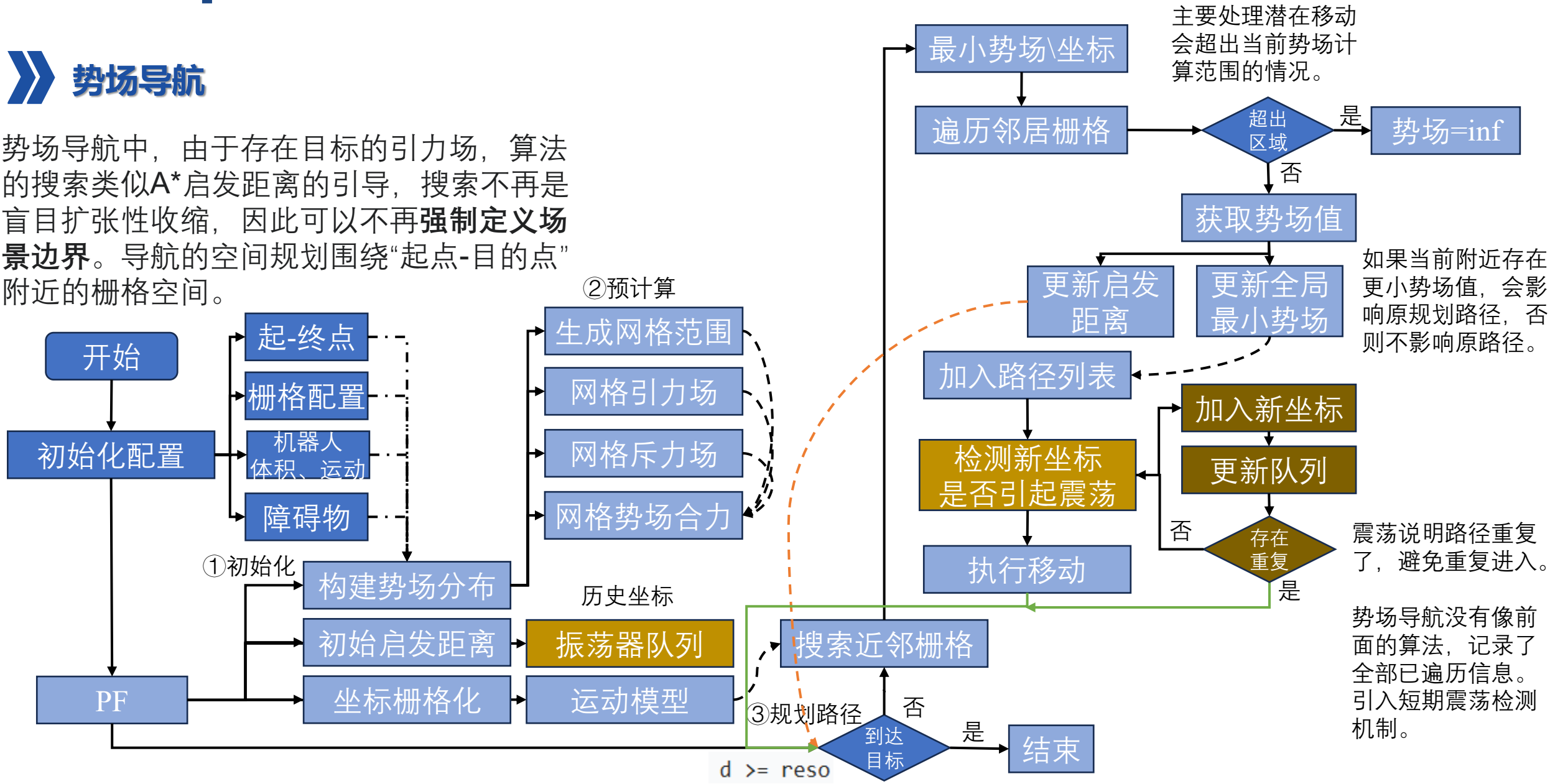
震荡检测：

防止机器人陷入局部最优，检测到重复行为和决策，强制朝一个方向。

Part 5 | Potential Field algorithm

势场导航

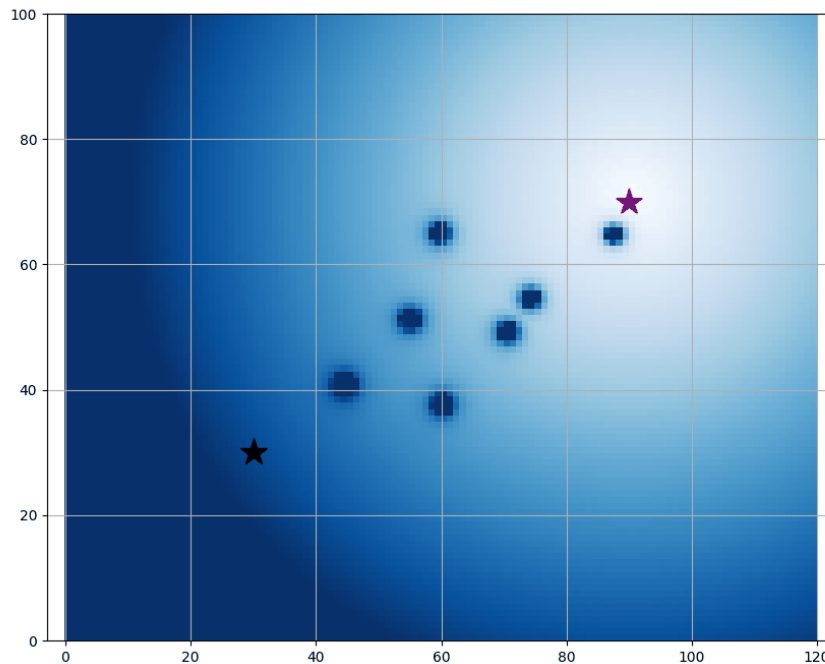
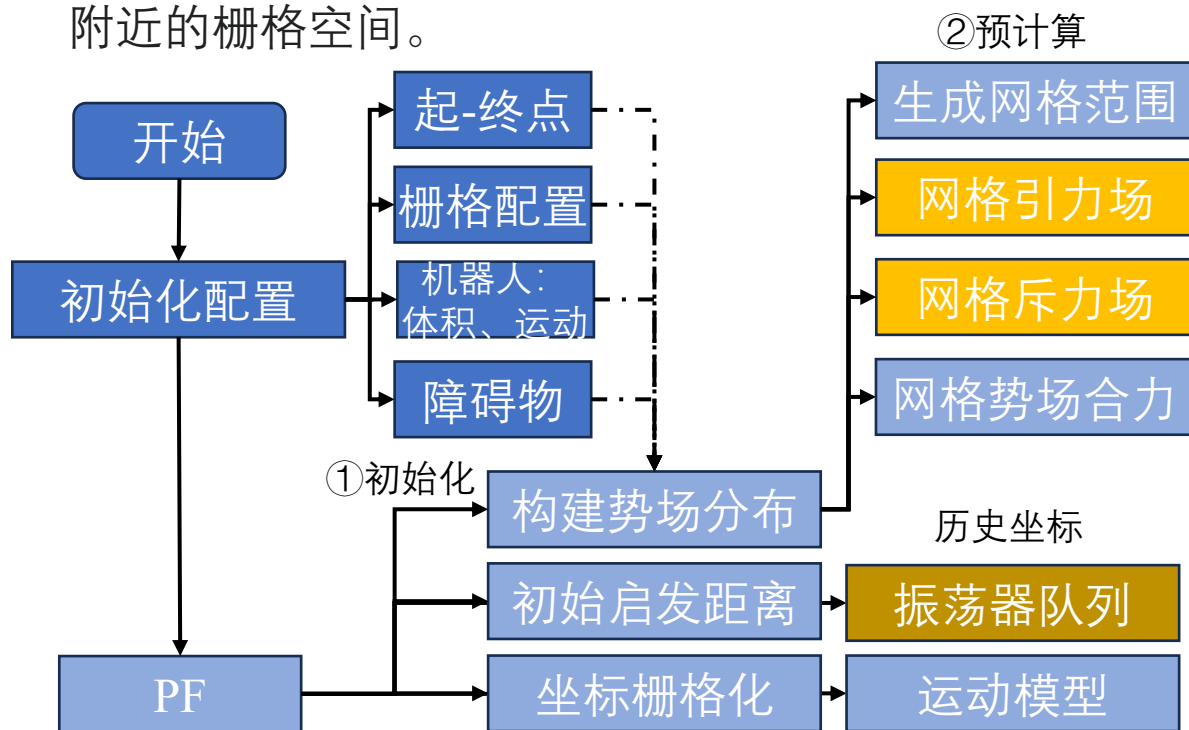
势场导航中，由于存在目标的引力场，算法的搜索类似A*启发距离的引导，搜索不再是盲目扩张性收缩，因此可以不再**强制定义场景边界**。导航的空间规划围绕“起点-目的点”附近的栅格空间。



Part 5 | Potential Field algorithm

» 势场导航

势场导航中，由于存在目标的引力场，算法的搜索类似A*启发距离的引导，搜索不再是盲目扩张性收缩，因此可以不再**强制定义场景边界**。导航的空间规划围绕“起点-目的点”附近的栅格空间。



因为只有一个终点，引力计算相对简单。
距离越远，引力越大： $U_g = 0.5 * KP * \|(x,y)-(gx,gy)\|^2$

```
return 0.5 * KP * np.hypot(x - gx, y - gy)
```

斥力可能来自多个障碍物，为简化计算，仅考虑离当前目标最近的一个，且考虑引力场作用范围。

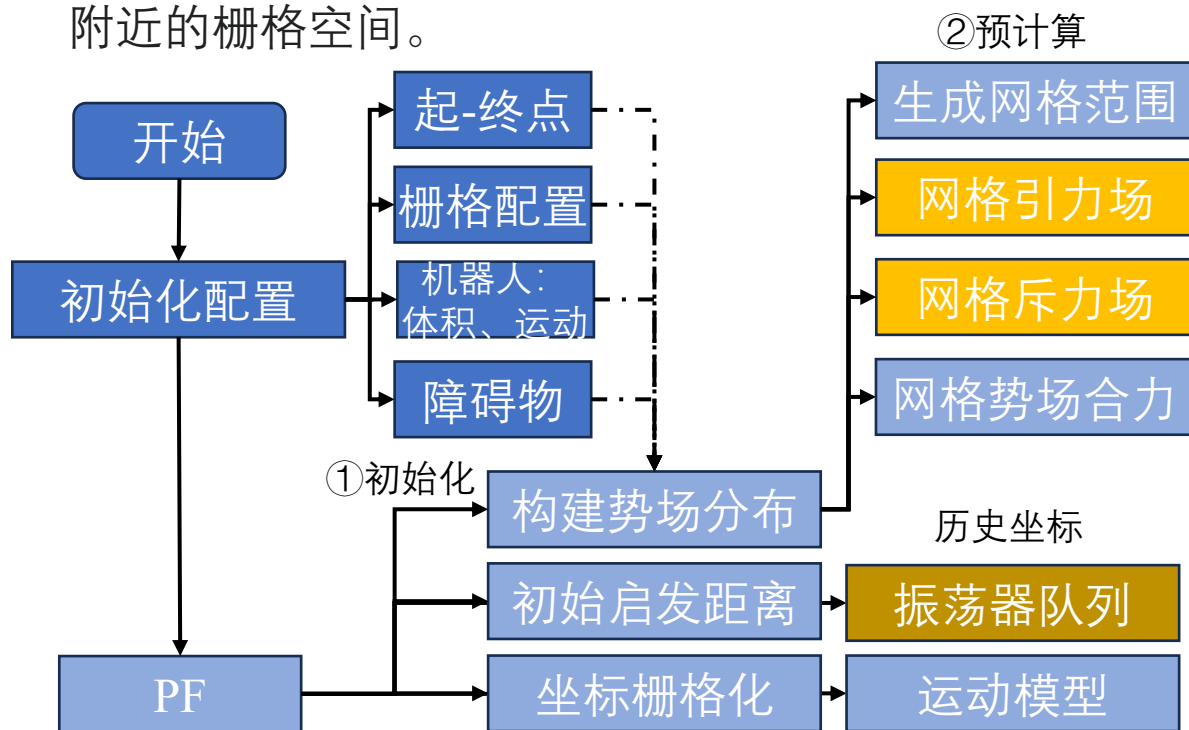
1. 确认最近障碍物；
2. 计算斥力，并基于阈值和作用范围

```
0.5 * ETA * (1.0 / min_distance - 1.0 / rr) ** 2
```

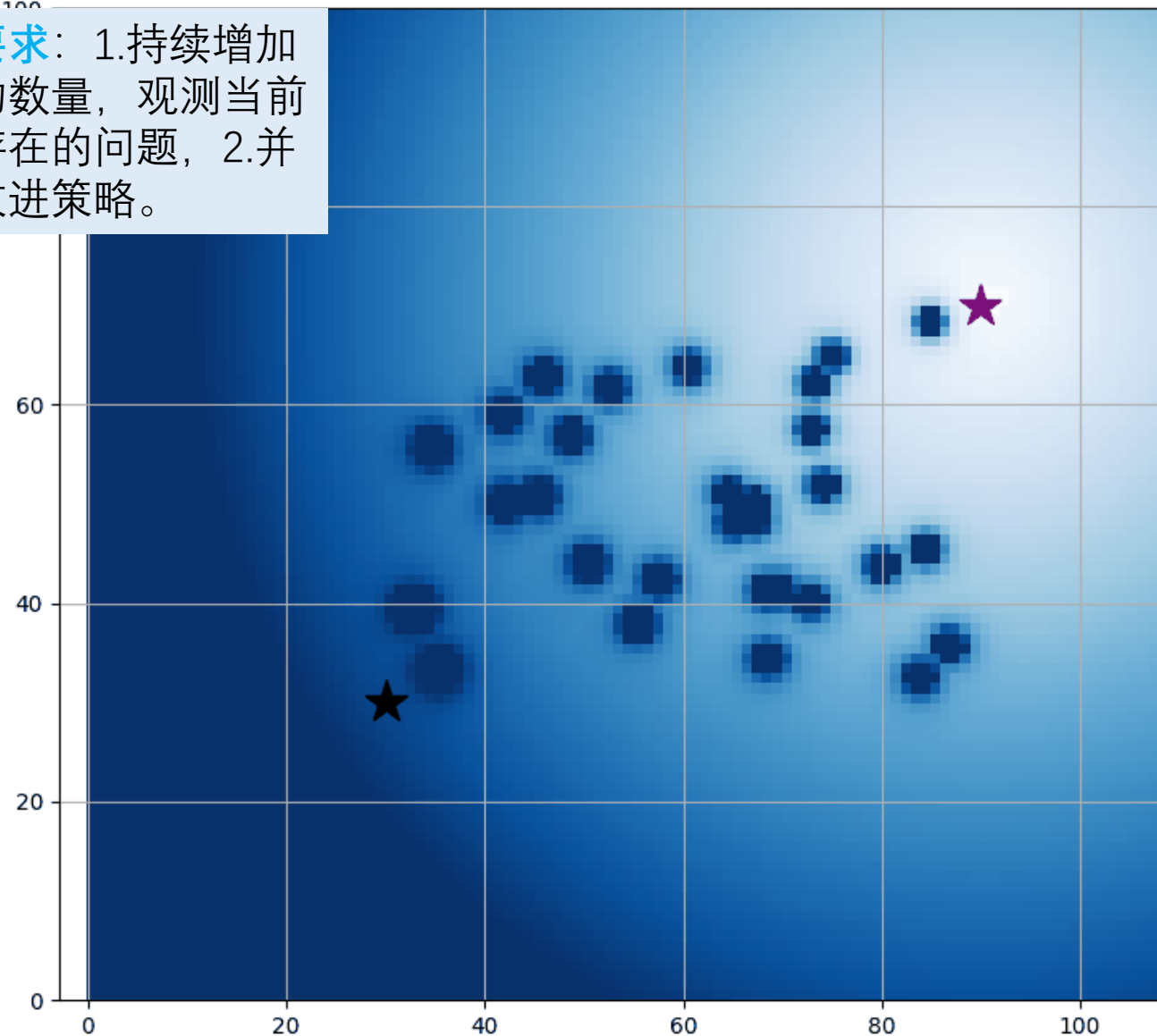

Part 5 | Potential Field algorithm

势场导航

势场导航中，由于存在目标的引力场，算法的搜索类似A*启发距离的引导，搜索不再是盲目扩张性收缩，因此可以不再**强制定义场景边界**。导航的空间规划围绕“起点-目的点”附近的栅格空间。



实验要求： 1.持续增加障碍物数量，观测当前算法存在的问题， 2.并考虑改进策略。



Part 5 | 总结

- 绝大多数的运动载体，可以简化到（局部）2D平面下的规划。
- DFS和BFS恰好可以看作是两个空间搜索的极端
- BFS希望构建全局度量，搜索全面，然后确定最佳路径。
- DFS希望沿着单一方向深入搜索，最佳和最坏情况差异较大。
- Dijkstra在其基础上引入最小移动代价，A*进一步引入启发距离，PF则是加入了对障碍物的碰撞避免。
- D*在A*基础上引入增量更新策略，适合动态环境（代码未体现）。
- 上述算法，或通过全面搜索（查全），或采用单一路径搜索（高效）的方式提升规划效率，虽然引入了一些启发条件，但在极端情况下容易失败。

开始实验



北京航空航天大学
BEIHANG UNIVERSITY